

Efficient Fuzzy PSI under One-Sided Assumptions

Artifact Appendix — CCS 2026, Paper [TODO: paper ID]

Xinpeng Yang, Meng Hao, Yanxue Jia, Chenkai Weng, Yonggang Wen, Tianwei Zhang

A ARTIFACT APPENDIX

A.1 Abstract

Our artifact provides the C++ implementation of fuzzy private set intersection (FPSI) under the unique-cell and unique-block assumptions, including normal and prefix-optimized protocols for L_∞ , L_1 , and L_2 distances. It includes build scripts, generated-input correctness tests, performance benchmarks, and paper-reference data. Evaluation produces raw logs, Markdown summaries, and SVG runtime comparisons with the *Ours/Ours-Px* entries in Tables 2 and 3. Our code is MIT-licensed; dependencies retain their respective licenses.

A.2 Description & Requirements

The root `README.md` is the entry point. Implementation modules are under `code/fpsi/`; `scripts/` contains categorized build, reproduction, and data tools. The `claims/` directory maps each claim to its commands, expected outputs, and paper-reference CSV files. Generated results are written to `artifact-results/`.

A.2.1 Security, privacy, and ethical concerns. Benchmarks use synthetic, non-sensitive points and execute locally. Installation downloads dependencies and may install system packages with root or sudo; benchmarks do not require either. Large cases can exhaust RAM; use an otherwise idle host with the memory budget below. This is a research prototype, not a production security implementation: the shared-output OPRF uses fixed benchmark keys and pre-constructed key-OT inputs. Tests validate functional outputs and performance, not deployment security.

A.2.2 How to access. Download: [TODO: download URL]. Permanent archive: [TODO: archival DOI]. Evaluated version: [TODO: release / commit]. Contact: [TODO: contact email].

A.2.3 Hardware dependencies. Use an x86-64 CPU supporting AES, PCLMUL, SSE2, and SSE4.1; ARM64 and Apple Silicon are unsupported. No GPU or second host is needed: both protocol parties run as local threads in one process. Build and basic validation require 16 GiB RAM. For full evaluation, recommend 256 GiB RAM; observed peak process RSS is approximately 63 GiB for unique-cell and 199 GiB for unique-block. Budgets below use an AMD EPYC 9554 host and one trial; slower hosts require more time. Leave at least 10 GiB free disk space for dependencies, build products, and results.

A.2.4 Software dependencies. The reference platform is Ubuntu 24.04; GCC 11 or newer with C++20 is required. Build scripts fetch pinned dependencies. Bash, CMake, Git, Python 3, and the system packages listed in `scripts/build/run.sh` are required. Internet access is needed for a fresh build. Result processing and SVG plotting use Python’s standard library and our modules only; Matplotlib is not required. Docker is the recommended deployment method.

A.2.5 Benchmarks. No external datasets or models are required. Benchmark inputs are generated unsigned-integer point sets satisfying the selected one-sided assumption. Performance scripts explicitly plant 16 matches (`-inter 16`). Correctness tests generate edge-case datasets and reference answers.

A.3 Set Up

A.3.1 Installation. Obtain the source archive, extract it, and run from its root:

```
./scripts/build/preflight.sh
./scripts/build/run.sh
```

Allow about 5 person-minutes and 10–20 compute-minutes for a fresh build on an 8-core machine. Preflight checks the host without modifying it; on a fresh host, install any missing prerequisites reported and rerun it. A successful build creates `code/build/fpsi` and reports `Build complete`. If system packages are already installed, use `FPSI_SKIP_SYSTEM_PACKAGES=1` with the build command to avoid package installation. Dependency versions and troubleshooting are in the root `README`.

A.3.2 Basic test. Run `./scripts/reproduction/run.sh --quick` (about 1 person-minute, under 2 compute-minutes, 16 GiB RAM). The smoke test must report `PASS: 6 protocol cases and 1 parameter guard`. Inspect `artifact-results/quick.txt` for the raw output and `summary.md` for measurements. This is an installation check, not the complete correctness experiment.

A.4 Evaluation Workflow

A.4.1 Major claims.

- (C1) The implemented protocols correctly compute FPSI by returning all sender elements that are close to the receiver set (E1).
- (C2) The receiver-sided unique-cell L_∞ implementations reproduce the runtime and communication trends of the *Ours/Ours-Px* entries in Table 2 (E2).
- (C3) The receiver-sided unique-block implementations reproduce the corresponding trends for L_∞ , L_1 , and L_2 in Table 3 (E3).

These experiments do not rerun competing baselines or establish security proofs. The performance claims concern our table entries, not independent reproduction of every baseline comparison.

Reported seconds use each protocol’s original timed interval, not process wall time: synthetic input generation, sockets, and mode-specific local preprocessing are excluded; OPRF, OKVS, MPC, final transfer, and enabled correctness checks are timed. The exact per-mode split is documented in `claims/paper-comparison.md`. Communication is the sum sent and received by one local endpoint, divided by 2^{20} bytes (labeled MB in the tables). Default runs do not apply network shaping.

A.4.2 Experiments. Run E1–E3 sequentially from the artifact root after installation. Budgets exclude build time and use the default `TRIALS=1`. Each wrapper exits nonzero on validation failure and preserves diagnostic logs. Increase `TRIALS`, e.g., `TRIALS=3` `bash claims/claim2/run.sh`, to obtain internal-trial mean measurements; budget proportionally more compute time.

E1: Correctness (C1). 5 person-minutes + 3–5 compute-minutes; 16 GiB RAM. **Preparation:** No input download is needed. **Execution:** Run `bash claims/claim1/run.sh`. **Results:** The wrapper runs smoke checks and 240 applicable dataset/configuration executions covering every implemented protocol variant. Cases include zero/full intersections, many-to-one matches, distances $\delta - 1$, δ , and $\delta + 1$, and cell/block-boundary matches. An independent Python integer-distance oracle checks every recovered sender point, not just the match count. The final report must include 240 executions across 16 modes; 0 failures. All 240 rows in `artifact-results/claim1/boundary/runs/summary.csv` must be PASS; per-case inputs, expected answers, outputs, and logs remain available.

E2: Unique-cell (C2, Table 2). 10 person-minutes + 3–4 compute-hours; 80 GiB RAM recommended. **Preparation:** Reserve an otherwise idle host. **Execution:** Run `bash claims/claim2/run.sh`. Both modes cover $n = m \in \{2^8, 2^{12}, 2^{16}\}$, $d \in \{2, 4, 6\}$, and $\delta \in \{32, 64, 128, 256, 512\}$: 90 configurations. **Results:** Under `artifact-results/claim2/`, expect 90 rows in both `summary.md` and `paper-comparison.md`, and nine runtime SVGs in `paper-plots/`, one per (n, d) pair. The terminal reports `Claim 2 full complete`. For smaller runs, use the shared partial reproduction or mini benchmark described in Notes on Reusability.

E3: Unique-block (C3, Table 3). 10 person-minutes; 2–3 hours compute; 256 GiB RAM recommended. **Preparation:** Reserve the full memory budget, especially for normal mode at $d = 6$. **Execution:** Run `bash claims/claim3/run.sh`. This evaluates $n = m = 2^{12}$, the same dimensions and thresholds as E2, all three metrics, and both modes: 90 configurations. **Results:** Under `artifact-results/claim3/`, expect 90 rows in each `summary/comparison` Markdown table and nine runtime SVGs, one per metric/dimension pair. The terminal reports `Claim 3 complete`.

For E2/E3, each trial must report Total 16/16 matches found!. Wrappers reject missing or mismatched configurations. Open each `paper-comparison.md` to inspect measured and paper runtime curves against equally spaced δ categories, in seconds with automatically scaled linear time axes. Communication should be stable for fixed parameters; absolute runtime varies with hardware. Assess scaling across thresholds, dimensions, and set sizes; do not require exact times or strictly increasing adjacent samples. The plots support manual interpretation, not an automatic trend-pass verdict.

A.5 Notes on Reusability

For lower-cost exploration, use `./scripts/reproduction/run.sh --partial`: 80 original-paper configurations at $n = 2^{12}$, $d \in \{2, 4\}$, about 25–35 minutes and 35 GiB peak RSS (64 GiB RAM recommended). The `--mini` profile uses the same matrix at $n = 2^{10}$: about 6–10 minutes and 21 GiB peak RSS (32 GiB RAM). Mini is not

a reproduction of paper set sizes. These profiles produce logs and summaries; use E2/E3 for automatic paper-comparison plots.

Custom data can be supplied using `-i <directory>` with `sender_data.txt` and `recver_data.txt`: one point per line, whitespace-separated unsigned coordinates, equal set sizes and dimensions, no duplicates within either set, and the selected input assumption satisfied. Matching sender points are written to `output.txt` in that directory. Without `-i`, simulated inputs are generated. The reusable boundary generator and modular protocol/primitive code are documented in `claims/claim1/claim.md` and `code/README.md`.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.